Retrieval-Augmented Generation (RAG) for the Global Workflow Knowledge Base

A Researcher's Guide to How AI Finds Answers
in 200,000+ Documents

NOAA EMC — Environmental Modeling Center
MCP/RAG Platform Team

May 2026

Contents

1 What is RAG?

Retrieval-Augmented Generation (RAG) is a technique that gives an AI assistant access to a curated library of documents *at the moment it answers your question*. Instead of relying solely on what the model memorized during training, RAG retrieves the most relevant passages from a knowledge base and feeds them to the language model as context.

Think of it this way: a language model without RAG is like a researcher working from memory alone. A language model *with* RAG is like that same researcher who can instantly pull the right page from a 200,000-document library before answering.

1.1 Why RAG Matters for Scientific Computing

Large language models are trained on broad internet data. They know general programming patterns and common scientific concepts, but they do *not* know:

- The specific structure of the NOAA Global Workflow
- Which Fortran subroutine handles vortex initialization in HAFS
- What environment variables `JGFS_FORECAST` expects
- How ESMF couples the atmosphere to the ocean via NUOPC
- The correct `err_chk` pattern for NCO production scripts

RAG bridges this gap by storing all of that domain knowledge in a searchable format and retrieving it on demand.

1.2 The Three Steps of RAG

Every RAG system follows the same fundamental pattern:

1. **Ingest** — Documents are broken into chunks, converted to numerical vectors (embeddings), and stored in a searchable database.
2. **Retrieve** — When a question arrives, the system converts the question to the same vector format and finds the most similar document chunks.
3. **Generate** — The retrieved chunks are passed to the language model as context, and the model generates an answer grounded in that evidence.

The quality of a RAG system depends primarily on Step 1 (what you put in) and Step 2 (how well you find it). Step 3 is handled by the language model itself.

2 How RAG Connects to MCP

The Model Context Protocol (MCP) is the transport layer — it defines how an AI assistant communicates with external tools and data sources. RAG is one of those data sources.

In our system, the MCP server exposes tools like `search_documentation` and `explain_with_context`. When the AI assistant needs domain knowledge, it calls these tools via MCP. Behind the scenes, the tool performs the RAG retrieval (Step 2) and returns the relevant passages. The assistant then uses those passages to formulate its answer (Step 3).

User Question → AI Assistant → MCP Tool Call → RAG Retrieval → Answer with Evidence

The key insight: MCP handles the *communication*, RAG handles the *knowledge*. They are complementary layers, not alternatives.

3 Embeddings: How Text Becomes Searchable

The core technology that makes RAG work is *embedding* — the process of converting text into a numerical vector (a list of numbers) that captures its meaning.

3.1 What is an Embedding?

An embedding is a fixed-length list of numbers (a vector) that represents the *semantic meaning* of a piece of text. Two pieces of text that mean similar things will have vectors that are close together in this numerical space, even if they use completely different words.

For example:

- "FV3 dynamical core" and "cubed-sphere atmospheric solver" would have *similar* vectors (same concept, different words)
- "FV3 dynamical core" and "chocolate cake recipe" would have *distant* vectors (unrelated concepts)

This property — that meaning maps to proximity in vector space — is what allows RAG to find relevant documents even when the user's question doesn't use the exact same terminology as the source material.

3.2 Amazon Titan Text Embeddings v2

Our knowledge base uses **Amazon Titan Text Embeddings v2**, a sentence transformer model provided by AWS through the Bedrock service. Key characteristics:

- **1024 dimensions** — each text chunk is represented as a vector of 1024 numbers
- **8,192 token input limit** — can process chunks up to approximately 32,000 characters
- **Multilingual** — handles technical English, code snippets, and mixed content
- **Managed service** — runs on AWS infrastructure, no local GPU required

When we ingest a document, each chunk of text is sent to Titan via the AWS Bedrock API. Titan returns a 1024-number vector that we store alongside the original text in OpenSearch.

3.3 Why We Switched to Titan

Previously, the system used a locally-hosted sentence transformer model (MPNet, 768 dimensions). The migration to Titan provided:

1. **Higher dimensionality** (1024 vs 768) — more capacity to distinguish subtle semantic differences
2. **No local compute** — embedding generation runs on AWS, freeing the server for other work
3. **Consistent quality** — AWS maintains and updates the model; we benefit from improvements without re-deploying
4. **Scale** — Titan handles our 200,000+ document corpus without memory constraints

The MPNet indices remain available as a fallback (the system maintains both), but all new ingestion targets the Titan 1024-dimensional indices.

4 The Vector Store: OpenSearch

Embeddings need a home — a database optimized for finding the nearest vectors to a query vector. We use **Amazon OpenSearch Service** with its k-NN (k-nearest neighbors) plugin.

4.1 How Vector Search Works

When you ask a question:

1. Your question is embedded using the same Titan model
2. OpenSearch finds the k document chunks whose vectors are closest to your question's vector (using cosine similarity)
3. Those chunks are returned as search results, ranked by similarity score

This is fundamentally different from keyword search. A keyword search for "atmosphere model" would miss documents that say "atmospheric solver" or "FV3 dynamical core." Vector search finds all of them because their *meanings* are close in embedding space.

4.2 Our Index Structure

The knowledge base is organized into five logical collections, each stored as a physical OpenSearch index:

Index	Documents	Content
mdc-workflow-docs-titan1024	30,173	Documentation from 53 URL sources
mdc-code-context-titan1024	90,135	Fortran/Shell/Python code with context
mdc-jjobs-titan1024	751	J-Job script headers and documentation
mdc-community-summaries-titan1024	2,113	Graph-derived subsystem summaries
mdc-ee2-standards-titan1024	34	EE2/NCO production coding standards

Table 1: OpenSearch indices in the Titan 1024-dim embedding space

Total: over 123,000 documents in the Titan indices alone, with an additional 85,000+ in the legacy MPNet indices for backward compatibility.

5 The Unified Ingest Manifest

The **unified ingest manifest** is the single source of truth for what documentation has been ingested into the knowledge base. It lives at:

`mcp_server_python/src/config/unified_manifest.json`

5.1 What the Manifest Tracks

For every documentation source in the knowledge base, the manifest records:

- **name** — unique identifier (e.g., `mom6`, `esmf-user-guide`)
- **source_type** — how the content is obtained (`url_crawl`, `code_parse`, `pdf_download`, etc.)
- **collection_target** — which OpenSearch collection receives the documents
- **embedding_profile** — which embedding model to use (`titan1024`)
- **url** — the documentation website to crawl
- **doc_count** — how many chunks are currently indexed
- **last_ingested** — when the source was last successfully crawled
- **tier** — priority classification for ingestion ordering
- **enabled** — whether the source is active

5.2 Source Types

The manifest supports seven source types, reflecting the diverse nature of the global workflow's documentation:

`url_crawl` Web documentation sites (ReadTheDocs, GitHub Pages, wikis). The crawler follows links within the site up to a configured page limit.

`pdf_download`
PDF reference documents. Downloaded, text-extracted via pypdf, chunked, and embedded.

`on_disk_submodule`
Local RST/Markdown files from the global-workflow git submodule tree.

`code_parse`
Source code (Fortran, Shell, Python) parsed at function boundaries with surrounding context.

`config_parse`
Configuration files (Rocoto XML, bash key-value configs) parsed with structure-aware chunking.

`standards`
EE2/NCO coding standards from the nws-hpc-standards repository.

`community_summary`
Graph-derived community summaries produced by the Leiden algorithm over the code graph.

5.3 Tier System

Sources are classified into five priority tiers that determine ingestion order and operational importance:

Tier	Sources	Description
tier1_critical	9	Core workflow, DA, coupling infrastructure (ESMF, NUOPC, GSI, global-workflow, UFS-Utils)
tier2_workflow	5	Workflow orchestration (ecFlow, Rocoto, wxflow, pyflow, uwtools)
tier3_models	18	Model components — atmosphere, ocean, ice, land, chemistry, waves (MOM6, CICE, FV3, HAFS, MPAS, etc.)
tier4_build	15	Build systems, I/O libraries (spack, NCEPLIBS, wgrib2, Kokkos)
tier5_standards	6	Coding standards and style guides (PEP8, Fortran best practices, METplus)

Table 2: Documentation source tier classification

5.4 Gap Detection

The manifest system includes automated gap detection that compares declared sources against live OpenSearch data. For each collection, it reports:

- **Coverage percentage** — actual documents vs. declared count
- **Never-ingested sources** — entries with no `last_ingested` timestamp
- **Stale sources** — entries not refreshed in 30+ days

This allows operators to quickly identify which documentation sources need attention without manually querying OpenSearch.

6 The Ingestion Pipeline

Getting documents from their source into the vector store involves a multi-step pipeline.

6.1 HTML Documentation Crawling

For web-based documentation (the majority of sources), the pipeline:

1. Reads the manifest to identify enabled sources
2. Crawls each URL, following internal links up to `max_pages`
3. Strips HTML to extract clean text content
4. Splits text into semantic chunks (respecting paragraph and section boundaries)
5. Sends each chunk to Amazon Titan for embedding (1024-dim vector)
6. Indexes the chunk + vector + metadata into OpenSearch
7. Updates the manifest with the new `doc_count` and `last_ingested` timestamp

6.2 PDF Document Ingestion

For PDF-only reference documents (like the ESMF Fortran API reference), a dedicated pipeline:

1. Downloads the PDF from the manifest URL
2. Extracts text page-by-page using pypdf
3. Chunks at 512-token windows with 64-token overlap between consecutive chunks
4. Embeds each chunk with Titan (same 1024-dim space as HTML docs)
5. Indexes with metadata including page number for traceability

The 512-token chunk size with 64-token overlap ensures that:

- Each chunk is well within Titan's 8,192-token input limit
- Overlapping windows prevent information loss at chunk boundaries
- Retrieval returns coherent passages rather than fragments

6.3 Code Ingestion

Source code (90,000+ Fortran subroutines, shell scripts, and Python modules) is ingested with function-boundary chunking — each chunk corresponds to a complete function or subroutine with its surrounding context (imports, module declarations, neighboring functions). This ensures that code search results are self-contained and actionable.

7 Search: Putting It All Together

When a researcher asks a question through the MCP interface, the `search_documentation` tool performs a hybrid search:

1. **Semantic search (k-NN)** — the question is embedded with Titan and the nearest vectors are found in OpenSearch
2. **Keyword search (BM25)** — traditional text matching catches exact terms the embedding might generalize over
3. **Reciprocal Rank Fusion (RRF)** — results from both methods are merged using a rank-based scoring formula that preserves the best of each approach
4. **Graph enrichment** (optional) — each result is annotated with the count of related entities in the code graph, helping identify which results connect to the broader system

This hybrid approach means the system finds relevant results whether the user asks in natural language ("how does the ocean couple to the atmosphere?") or with specific technical terms (`ESMF_FieldBundleCreate`).

8 Current State of the Knowledge Base

As of May 2026, the knowledge base contains:

- **209,277 total documents** across all indices
- **30,173 documentation chunks** in the Titan workflow-docs index (from 53 URL sources + 4 PDFs)
- **90,135 code chunks** covering Fortran, Shell, and Python from the global-workflow source tree
- **105,891 graph nodes** and **2.9 million relationships** in the Neptune code graph
- **64 declared sources** in the unified manifest (55 url_crawl + 9 non-URL)

The system is healthy (4/4 components operational) and serves queries through both the AgentCore-hosted Python runtime and the local Node.js gateway.

9 Summary

RAG transforms a general-purpose AI assistant into a domain expert by giving it instant access to curated documentation at query time. The key components are:

1. **Embeddings** (Titan 1024-dim) convert text to searchable vectors
2. **OpenSearch** stores and retrieves those vectors efficiently
3. **The manifest** tracks what's been ingested and identifies gaps
4. **The ingest pipeline** keeps the knowledge base current
5. **MCP** provides the communication layer between the AI and the RAG system

The result: when a researcher asks about MPAS mesh generation or NUOPC component coupling, the system retrieves the actual documentation — not a hallucinated approximation — and grounds its answer in evidence.

A Appendix: Complete Source Inventory

The following tables list every documentation source in the unified ingest manifest as of May 20, 2026 (manifest version 9.0.0).

A.1 Tier 1 — Critical (9 sources)

Source	Docs	URL
esmf-user-guide	10,513	earthsystemmodeling.org/. . . /ESMF_usrdoc/
esmf-ref-pdf	1,165	earthsystemmodeling.org/. . . /ESMF_refdoc.pdf
esmc-ref-pdf	125	earthsystemmodeling.org/. . . /ESMC_crefdoc.pdf
nuopc-layer-ref	248	earthsystemmodeling.org/. . . /NUOPC_refdoc/
nuopc-ref-pdf	487	earthsystemmodeling.org/. . . /NUOPC_refdoc.pdf
esmpy-pdf	94	earthsystemmodeling.org/. . . /ESMPy.pdf
global-workflow	236	global-workflow.readthedocs.io/en/latest/
ufs-utils	129	noaa-emcufs-utils.readthedocs.io/en/latest/
gsi-user-guide	87	dtcenter.org/. . . /gsi/docs/users-guide/

A.2 Tier 2 — Workflow (5 sources)

Source	Docs	URL
ecflow	338	ecflow.readthedocs.io/en/latest/
pyflow	103	pyflow-workflow-generator.readthedocs.io/
rocoto	74	christopherwharrop.github.io/rocoto/
uwtools	152	uwtools.readthedocs.io/en/latest/
wxflow	95	wxflow.readthedocs.io/en/latest/

A.3 Tier 3 — Models (18 sources)

Source	Docs	URL
catchem	45	ufs-community.github.io/CATChem
cdeps	62	escomp.github.io/CDEPS/
cece	38	ufs-community.github.io/CECE
cice	325	cice-consortium-cice.readthedocs.io/
cmaq	403	github.com/USEPA/CMAQ/wiki
cmeps	0	escomp.github.io/CMEPS/
fms	933	github.com/NOAA-GFDL/FMS/wiki
fv3-docs	851	github.com/NOAA-GFDL/. . . /wiki
gocart	1,025	geos-chem.readthedocs.io/
hafs	78	hafsdoc.readthedocs.io/en/latest/
jedi-docs	107	jedi-docs.readthedocs-hosted.com/
land-da	56	land-da.readthedocs.io/en/stable/
mom6	901	mom6.readthedocs.io/en/main/
mpas-atmosphere	134	www2.mmm.ucar.edu/projects/mpas/

Source	Docs	URL
pyioda	142	jedi-docs. . . /ioda/index.html
ufs-srweather-app	189	ufs-srweather-app.readthedocs.io/
ufs-weather-model	356	ufs-weather-model.readthedocs.io/
ww3-wiki	2,282	github.com/NOAA-EMC/WW3/wiki

A.4 Tier 4 — Build / Libraries (15 sources)

Source	Docs	URL
ccpp-techdoc	214	ccpp-techdoc.readthedocs.io/
kokkos-api	0	kokkos.org/kokkos-core-wiki/
kokkos-overview	49	kokkos.org/about/overview/
nceplibs-bacio	95	noaa-emc.github.io/NCEPLIBS-bacio/
nceplibs-bufr	1,251	noaa-emc.github.io/NCEPLIBS-bufr/
nceplibs-g2	388	noaa-emc.github.io/NCEPLIBS-g2/
nceplibs-g2tmpl	158	noaa-emc.github.io/NCEPLIBS-g2tmpl/
nceplibs-ip	838	noaa-emc.github.io/NCEPLIBS-ip/
nceplibs-nemsio	9	noaa-emc.github.io/NCEPLIBS-nemsio/
nceplibs-sfcio	0	noaa-emc.github.io/NCEPLIBS-sfcio/
nceplibs-sigio	1	noaa-emc.github.io/NCEPLIBS-sigio/
nceplibs-w3emc	648	noaa-emc.github.io/NCEPLIBS-w3emc/
spack	1,969	spack.readthedocs.io/en/latest/
spack-stack	162	spack-stack.readthedocs.io/
wgrib2	366	cpc.ncep.noaa.gov/. . . /wgrib2/

A.5 Tier 5 — Standards (6 sources)

Source	Docs	URL
fortran-best-practices	876	fortran-lang.org/learn/best_practices/
google-shell-style	36	google.github.io/styleguide/shellguide.html
metplus	444	metplus.readthedocs.io/en/latest/
numpy-docstrings	31	numpydoc.readthedocs.io/
pep8	42	peps.python.org/pep-0008/
upp	198	upp.readthedocs.io/en/latest/

A.6 Non-URL Sources (9 sources)

Source	Type	Docs	Description
fortran-code-context	code_parse	77,613	Fortran subroutines/functions
shell-code-context	code_parse	2,079	Shell scripts (j-jobs, ush)
python-code-context	code_parse	8,922	Python modules/utilities
rocoto-config	config_parse	187	Rocoto XML workflow defs

Source	Type	Docs	Description
expdir-configs	config_parse	1,297	Experiment config files
global-workflow-rst	on_disk	1,759	Local RST documentation
ee2-standards	standards	34	EE2/NCO coding standards
community-summaries	community	2,113	Graph-derived summaries
jjob-docs	jjob_docs	751	J-Job script headers